Secret Rotation and History Purge Runbook

Companion to KEY_AND_KEYSTORE_STORAGE_STANDARD.md. Use this runbook for any secret, key, or keystore password that was committed to TFVC or Bitbucket, published to JFrog, written to a log, or otherwise exposed.

> Never paste a secret value into a command line, ticket, PR, chat, log, or this repository. Commands below read values from Vault or from a variable filled by a secure prompt.

---

1. Order of operations

Rotation comes first. Purging history without rotating first leaves a live credential in every existing clone, artifact, and backup.

Step	Action	Owner	Evidence (no values)
1	**Inventory** every location of the exposed value: repos and branches, forks and mirrors (including the `fis-anthropic-poc` evidence mirror), artifacts, host files, logs, tickets. Use Checkmarx 2ms / gitleaks across full history with values redacted	Engineer	Scan report IDs, list of locations
2	**Create the new secret per tier** in Vault (`secret/ibsmas/<tier>/<repo>/<name>`). Generate it in Vault, or on a hardened admin host with an immediate write to Vault	Secret owner	Vault path + KV version number per tier
3	**Deploy the consumer change** (code that reads the new source and fails closed) to System, and validate against the System system-test plan	Engineer	Harness execution ID, test result
4	**Re-encrypt or regenerate dependent artifacts** per tier (`EncPwd`, `B04SecretKey` ciphertext, UserMaintenance blobs, keystores) from the new key	Secret owner	Per-tier confirmation
5	**Cut over** Readiness → Prod → DR through Harness with CAB approval	Ops	Change record, Harness execution IDs
6	**Revoke the old value**: change the DB password, re-key the certificate and revoke the old one, rotate the trusted-auth secret in the BOE CMC, rotate the Voltage credential	Secret owner	Revocation timestamp
7	**Purge** source history (§3), artifacts (§4), and logs (§5)	Engineer + admins	Purge ticket, before/after scan
8	**Verify**: rescan history and artifacts, confirm that the old value fails authentication, and run the Mythos rescan on `master`	Engineer	Rescan result

Rollback rule: if a cut-over fails, fix provisioning forward. Never restore a literal to source or to an artifact. Any rollback that reintroduces a literal is a security incident and reopens the finding.

2. Rotation by secret type

Secret type	Examples	Rotate by	Notes
Oracle account password	DWNN (f014), bodw/bocrd (f080), RS `BaseDbContext`	DBA `ALTER USER`, new value written to Vault (or CyberArk auto-rotation)	Review grants at the same time (f014)
Keystore password + private key	BOE Tomcat `*.pfx` (f012)	New CSR → new cert from the tier CA → new PKCS#12 or Windows-store import (non-exportable) → revoke the old cert	Changing only the password does **not** help if the old `.pfx` and password were both exposed. Re-key
KEK / symmetric key	Voltage `AESKey` (f006), `BICSSO_BO4_KEK` (f010), `FIS_ENCRYPTION_KEY` (f001)	Generate in Vault (Transit `rotate`, or KV with ≥ 256-bit random), then re-encrypt all dependents	Rotate the protected secret too (Voltage credential, trusted-auth secret), because the old KEK decrypts the old ciphertext still in history
Shared secret	BOE trusted-auth secret (f010)	BOE CMC → Trusted Authentication → new shared secret; update Vault per tier	All 8 `bicsso-config` environments
RSA keypair (SSO channels)	SLAPI channel keys (f024)	Generate a **distinct** keypair per channel, and coordinate the public-key exchange with the token issuer	Split the shared keypair; track it as its own ticket
.NET RSA key container	`RsSecureKey` (RS)	New container per tier (`aspnet_regiis -pc ... -exp`), re-encrypt the sections, import on farm nodes, delete the export file	The export XML contains the private key. Store it in Vault only. Exportable by necessity; see the standard's §9.6 RS-2 exception
Pipeline credential	Artifactory token, Bitbucket token, Checkmarx client secret	Regenerate in the tool, update the Jenkins credential / Vault path	Use expiring scoped tokens

3. Source history purge

3.1 Bitbucket (git)

Preconditions: rotation steps 1–6 are done, a Bitbucket admin is engaged, all open PRs are listed, and the team has been told to stop pushing.

```
# Read the token from a secure prompt (never type it on a command line)
$token = [System.Net.NetworkCredential]::new('', (Read-Host -AsSecureString "Enter Bitbucket Token")).Password

# Pass the auth header through environment config (git 2.31+), not `git -c`,
# which would expose the token in the process list to other users on the host
$env:GIT_CONFIG_COUNT   = 1
$env:GIT_CONFIG_KEY_0   = 'http.extraHeader'
$env:GIT_CONFIG_VALUE_0 = "Authorization: Bearer $token"

# Fresh mirror clone
git clone --mirror <repo-url> repo.git
```

```
cd repo.git

# Rewrite history in ONE filter-repo run: filter-repo refuses a second run on a
# clone that is no longer fresh. Drop whichever option you do not need.
# --replace-text reads an expressions file kept OUTSIDE any repo and deleted afterwards.
git filter-repo --invert-paths --path src/main/resources/AESKey.txt --replace-text C:\secure\replacements.txt

# Verify with values suppressed
gitleaks detect --source . --redact --report-path ..\post-purge.json   # or the Checkmarx 2ms CLI

# filter-repo removes the origin remote by design; add it back before pushing
git remote add origin <repo-url>

# Push branches and tags only. `--mirror` would also try to rewrite Bitbucket's
# read-only refs/pull-requests/* and report a misleading partial failure.
git push --force origin 'refs/heads/*:refs/heads/*' 'refs/tags/*:refs/tags/*'

# Clear the token from the session. PSReadLine already skips saving lines that
# contain "token"; Clear-History only clears this session's in-memory history.
Remove-Item Env:GIT_CONFIG_COUNT, Env:GIT_CONFIG_KEY_0, Env:GIT_CONFIG_VALUE_0
Remove-Variable token -ErrorAction SilentlyContinue
```

After the push:

- A Bitbucket admin runs repository garbage collection / cache cleanup so unreachable objects are removed. PR diffs and comments can still show old content, so decline or delete the affected PRs with admin help.
- Every developer and every Jenkins workspace **re-clones**. Old clones reintroduce the history on push.
- Forks and mirrors (including the `fis-anthropic-poc` evidence mirror) need the same treatment or explicit risk acceptance.
- **Git LFS / Submodules:** If the repository tracks keystores (`*.pfx`, `*.jks`) or assets via **Git LFS**, verify whether LFS pointer files or actual LFS storage blobs hold the credentials. LFS storage requires separate server-side purging or administrative prune, as standard `filter-repo` rewrites git tree commits but does not automatically delete remote LFS storage objects.

3.2 TFVC

`tf destroy` needs the **Destroy** permission (project collection administrators).

```
# Preview
tf destroy "$/Project/Path/secret.file" /preview /collection:<url>
# Permanently remove item and all versions (content and history)
tf destroy "$/Project/Path/secret.file" /collection:<url>
# Remove content of old versions but keep the history record, e.g. keep from changeset N onward
tf destroy "$/Project/Path/secret.file" /keephistory /stopat:C<N> /collection:<url>
```

- A value embedded in a file that must stay (for example a `web.config`) cannot be removed line-by-line with `tf destroy`. Destroy the affected versions (`/keephistory /stopat`) or accept the history exposure, **after rotation**.
- Run `tf destroy ... /startcleanup` only with the collection admin. Otherwise the server cleanup job runs on its schedule.
- Shelvesets can hold the value too: `tf shelvesets /owner:*` and delete any affected ones.

4. JFrog Artifactory cleanup

1. Use AQL to find artifacts from affected builds: repos `ibsmas-*`, the build names of the affected Jenkins jobs, and dates before the fix merge.
2. Download to an isolated host and run the secret scan with values suppressed to confirm the hits.
3. Delete them, or move them to a restricted `quarantine-local` repository with no read permission for build users. Record the paths and checksums.
4. Empty the Artifactory trash can for the deleted paths (admin).
5. Turn on Xray / Advanced Security secrets detection so future uploads are flagged.

5. Logs and other stores

- Tomcat access logs containing `JSESSIONID` / `X-Sap-Logontoken` (f013): restrict ACLs, then purge or roll archives past retention. Tokens are session-bound, so invalidate sessions by restarting BOE services after deploy.
- Voltage trace logs (f079), UserMaintenance console capture (f080): find the scheduler/job logs and purge the affected lines or files.
- `catalina.<date>.log` if `-Dkeystore.pass` was ever used (`VersionLoggerListener`).
- Jira, Confluence, email, and SNOW attachments that quote the Mythos rationale with a secret value: redact them. `TRIAGE.json` rationale fields include one literal (f012, f014).
- Local `target/` directories and workspace evidence copies: `mvn clean` and delete them.

6. Evidence template (paste into the Jira ticket)

```
Finding: fNNN / AVIT########       Repo: <repo>
Inventory scan: <tool> <report id>  Locations: <count>
Vault paths (per tier): sys=<path@vN> rdy=<path@vN> prd=<path@vN> dr=<path@vN>
Consumer deployed: SYSTEM <harness exec id>, RDY <id>, PRD <id>, DR <id>
Old value revoked: <timestamp> by <role>
Source purge: <bitbucket/tfvc> <ticket>, post-purge scan <report id> clean
Artifact purge: <n> artifacts <deleted|quarantined>, AQL saved at <link>
Logs purge: <scope>
Mythos rescan on master: <RITM> <result>
```